Revision 37–41 · Paged KV memory

Interactive revision of lessons 37, 38, 39, 40 and 41: why contiguous KV breaks, blocks and block tables, PagedAttention, the block manager, and preemption under KV pressure.

Q1 · The slab bill PARTIAL

Covers: 17, 37

> **QUESTION**
>
> Llama 3.1 8B (32 layers, 8 KV heads, head dim 128, BF16) gets one contiguous KV slab per request, sized for max_model_len = 8,192 tokens. The KV pool is 50 GB and the average request uses 1,000 tokens. How many requests fit at once, what fraction of each slab holds real data, and how many would fit if only the used tokens were stored?

How I solved it

1. Bytes per token: K and V, times layers, KV heads, head dim and 2 bytes for BF16: $2 \times 32 \times 8 \times 128 \times 2 = 131{,}072$ B = 128 KiB.
2. The slab is reserved for the full 8,192 tokens: $8{,}192 \times 128$ KiB = 1 GiB ≈ 1.07 GB per request.
3. Requests that fit: 50 GB ÷ 1.07 GB ≈ 46.
4. Real data per slab: 1,000 ÷ 8,192 ≈ 12%.
5. Storing only what's used: 1,000 × 128 KiB ≈ 131 MB per request, so 50 GB ÷ 131 MB ≈ 380 requests, about 8× more.

Final answer: 46 requests, 12% real data, about 380 if only used tokens were stored.

> **RE-DRILL**
>
> Re-drill: rebuild 128 KiB per token from the config without notes, including the bytes per value.

Q2 · Refused with memory free PARTIAL

Covers: 37

> **QUESTION**
>
> A contiguous-slab engine has 3.6 GB free in total, scattered in gaps between live slabs. A new request needs a 2.2 GB slab and is refused. What is this waste called, what are the other two wastes of a contiguous slab, and why doesn't the engine just compact the slabs?

Answer

- **External fragmentation:** enough memory is free in total, but no single gap is big enough for the slab.

- **Reserved:** while a request runs, the space ahead of its real tokens is held for tokens it hasn't generated yet, and nobody else can use it.
- **Internal fragmentation:** the request finished early (a few hundred tokens), so the rest of its slab was never used at all.
- **No compaction:** closing the gaps means physically copying gigabytes of KV while requests are running, which is far too slow for a live server.

> **RE-DRILL**
>
> Re-drill: name all three wastes and say which one paging still leaves behind (internal, at most one partial block).

Q3 · Find the token PARTIAL

Covers: 38, 40

> **QUESTION**
>
> Blocks hold 16 tokens. A request's block table is [3, 11, 6, 20, 2]. Which physical block and slot hold token 70 (counting from 0)? Given exactly 5 table entries, what's the shortest and longest the request could be?

How I solved it

1. Divide by the block size: $70 = 4 \times 16 + 6$, so token 70 is in logical block 4, slot 6.
2. Look up logical block 4 in the block table (index 4): physical block 2.
3. So token 70's KV is in physical block 2, slot 6.
4. Lazy allocation only adds a block once the previous one is full, so 5 entries means the first 4 are full: at least $4 \times 16 + 1 = 65$ tokens.
5. At most all 5 blocks are full: $5 \times 16 = 80$ tokens.

Final answer: block 2, slot 6; between 65 and 80 tokens.

> **RE-DRILL**
>
> Re-drill: do one lookup end to end with the table, and state the length range for a given number of blocks.

Q4 · Pick the block size SOLID

Covers: 38, 39, 40

> **QUESTION**
>
> One teammate wants a block size of 1 token to wipe out tail waste; another wants 256 so block tables are short. What does each cost, and why is 16 a reasonable middle?

Answer

- **Block size 1:** no tail waste, but a 1,000-token request needs 1,000 table entries and 1,000 allocations, and every token is a scattered memory read, so the attention kernel loses coalesced reads within a block.
- **Block size 256:** short tables, but the tail can waste up to 255 tokens (about 25% of a 1,000-token request), and copy-on-write has to copy a much bigger block.
- **16:** tail waste stays at most 15 tokens (1.5% of 1,000) while tables and lookups stay small and reads within a block stay contiguous.

> **RE-DRILL**
> Re-drill: put numbers on both ends for a 1,000-token request (entries and worst-case waste).

Q5 · Slower kernel, faster server PARTIAL

Covers: 16, 39

> **QUESTION**
> An interviewer says PagedAttention reads scattered keys and values, so it must approximate attention, and its kernel is 20–26% slower than a contiguous one. How can vLLM claim 2–4× higher throughput?

Answer

- **Exact, not approximate:** attention needs every key and value in logical order, never physically adjacent. The kernel walks the block table, computes all scores, applies one softmax over all of them, then sums the values block by block. Only the addresses change.
- **Where the 20–26% comes from:** inside the kernel, a table lookup per block plus scattered reads at block boundaries (reads within a block stay coalesced).
- **Why it barely shows:** attention is only part of a decode step and decode is memory-bound, so a 26% slower attention kernel is about 6% on the step if attention is a quarter of it.
- **Where the 2–4× comes from:** near-zero waste fits far more requests, and a bigger batch spreads each weight read over many more tokens.

> **RE-DRILL**
> Re-drill: estimate step-level impact from kernel slowdown × attention's share of the step.

Q6 · Four samples, one prompt PARTIAL

Covers: 40

> **QUESTION**
>
> A user asks for n = 4 different answers to the same 1,000-token prompt, with 16-token blocks. How many physical blocks does the prompt need with and without sharing? When each answer writes its first new token, how many copies does copy-on-write make, and how many blocks are in use right after?

How I solved it

1. Prompt blocks: 1,000 = 62 × 16 + 8, so 63 blocks: 62 full and a last one holding 8 tokens.
2. With sharing: all 4 block tables point at the same 63 blocks, each with reference count 4.
3. Without sharing: 4 × 63 = 252 blocks.
4. First writes go into the partial last block. Writers 1, 2 and 3 each find it shared, copy it, and drop its count to 3, 2, then 1.
5. Writer 4 finds count 1 and writes in place, no copy.
6. Blocks in use: 63 + 3 copies = 66, against 252 without sharing.

Final answer: 63 vs 252 blocks; 3 copies; 66 blocks in use.

> **RE-DRILL**
>
> Re-drill: redo with a prompt that ends exactly on a block boundary (e.g. 1,024 tokens): how many copies then?

Q7 · The next chat turn PARTIAL

Covers: 40

> **QUESTION**
>
> A chat request finishes; 5 seconds later the same user sends the next turn (the whole conversation plus a new message). With vLLM's defaults, what happened to the finished request's blocks, and how can the new request skip recomputing most of them? Under heavy load, which old blocks are most likely already overwritten, and why?

Answer

- **On finish:** the manager walks the block table and decrements each count; blocks that reach 0 go to the back of the free queue.
- **Not wiped:** with prefix caching on (the default), a freed block keeps its keys and values, and each full block is labelled with a hash of its tokens.
- **Reuse:** the next turn's prefix hashes to the same values, so those blocks are pulled back out of the free queue and not recomputed.
- **What goes first:** new allocations take from the front of the queue (least recently used). A request's blocks are freed tail first, because the deep end of a conversation is the least likely to be reused, so the tail is closest to the front and is overwritten first.

- **Result under load:** the next turn reuses the start of the conversation and recomputes only the overwritten tail.

> **RE-DRILL**
>
> Re-drill: explain why a long system prompt shared by many users almost never gets overwritten.

Q8 · Coding: block manager PARTIAL

Covers: 38, 40, 41

> **QUESTION**
>
> Fill in `allocate`, `fork`, `append` and `free` for a block manager with a FIFO free queue, reference counts and copy-on-write (exercises/37_41_block_manager.py). Test 1: a 40-token prompt forked into 4 answers that each write one token. Test 2: a 32-token prompt (ends on a block boundary) forked once, then grown until the pool runs dry.

How I solved it

1. `allocate`: blocks needed = ceil(n ÷ 16), written as (n + 15) // 16; take that many from the front of the free queue.
2. `fork`: the child gets a copy of the parent's table (same physical blocks), and each block's count goes up by one.
3. `append`: if n is a multiple of 16 the last block is full, so take a fresh block; else if the last block's count is above 1, copy-on-write (fresh block, old count − 1, one copy); else write in place.
4. `free`: walk the table tail first, decrement each count, and put blocks at 0 on the back of the queue.

Results: test 1 uses 6 blocks after 3 copies (the 4th answer writes in place); freeing answer 1 returns only its own copy, block 3. Test 2 makes no copies, because the prompt ends on a block boundary and each first write needs a fresh block anyway; the 49th token raises MemoryError.

```
def allocate(self, seq, n_tokens):
    needed_blocks = (n_tokens + self.bs - 1) // self.bs
    self.tables[seq] = [self._take() for _ in range(needed_blocks)]
    self.lens[seq] = n_tokens

def fork(self, parent, child):
    self.tables[child] = list(self.tables[parent])   # copy the table, share the blocks
    self.lens[child] = self.lens[parent]
    for block in self.tables[child]:
        self.ref[block] += 1

def append(self, seq):
    t, n = self.tables[seq], self.lens[seq]
    if n % self.bs == 0:                 # last block full -> fresh block
        t.append(self._take())
    elif self.ref[t[-1]] > 1:            # partly full and shared -> copy-on-write
        old = t[-1]
        t[-1] = self._take()
        self.ref[old] -= 1
        self.copies += 1
    self.lens[seq] = n + 1               # otherwise write in place

def free(self, seq):
    for block in reversed(self.tables.pop(seq)):   # tail first
        self.ref[block] -= 1
        if self.ref[block] == 0:
            self.free_q.append(block)
    del self.lens[seq]
```

RE-DRILL

Re-drill: write `append` from memory with its three cases in order (full, shared, in place).

Q9 · How long until it runs dry? PARTIAL

Covers: 38, 41

QUESTION

The pool has 2,000 free blocks (16 tokens each) right after admission; 250 requests are decoding at 20 ms per step. How many steps and seconds until the free queue is empty, and why didn't the scheduler see it coming when it admitted them?

How I solved it

1. Each request adds one token per step and needs a new block every 16 tokens: 250 ÷ 16 = 15.6 blocks per step.
2. Steps until empty: 2,000 ÷ 15.6 = 128 steps.
3. Time: 128 × 20 ms ≈ 2.6 s.
4. At admission the scheduler only knew each request's current size (its prompt), not how long its answer would grow. Paging reserves nothing ahead, so the cost of future tokens arrives a few blocks per step.

Final answer: about 128 steps, about 2.6 seconds; admission only sees today's size.

RE-DRILL

Re-drill: redo it in tokens (2,000 × 16 = 32,000 free tokens ÷ 250 tokens per step = 128 steps) and notice the block size cancels out.

Q10 · Evict the biggest instead? PARTIAL

Covers: 39, 40, 41

QUESTION

A teammate wants to evict the request holding the most blocks when the pool runs dry, since that frees the most room at once. What's wrong with that, why does newest-first make sense, and why can't the engine free just half of a big request's blocks?

Answer

- **Biggest = most progress:** the biggest request is usually old and close to finishing, so evicting it throws away the most work and makes the recompute prefill the most expensive. Long requests could be evicted again and again and never finish.
- **Newest first:** it has the least progress, so it's the cheapest to bring back, and older requests keep their progress.
- **Sharing caveat:** evicting a request whose blocks are shared frees less than its size suggests, because shared blocks only lose one count.
- **All or nothing:** every new token's attention reads all earlier keys and values, so half a request is useless; the missing half would have to be recomputed before the next token anyway.

RE-DRILL

Re-drill: say what the victim costs to bring back with recompute, in terms of its total tokens (prompt + generated).

Q11 · Is a full cache a problem? PARTIAL

Covers: 20, 30, 41

QUESTION

KV cache usage has been pinned at 100% for an hour, and the preemption counter stayed flat at 0. Is the server in trouble? What would you need to see before calling it KV pressure, and what would users notice?

Answer

- **Not by itself:** 100% usage with 0 preemptions means running requests fill the pool and admission is holding new requests back. The server is busy, not sick.

- **Not reuse:** the usage figure counts blocks held by running requests; cached-but-free blocks count as free, so 100% doesn't mean lots of prefix reuse.
- **KV pressure looks like:** the preemption counter climbing, plus "preempted … because there is not enough KV cache space" warnings in the log.
- **Queue length** is a capacity signal: if the waiting queue grows, users see a longer TTFT, but answers that have started still stream smoothly.
- **Under real pressure** users see answers stall mid-stream while evicted requests wait and then recompute.

> **RE-DRILL**
>
> Re-drill: map each dashboard pattern to a user symptom: queue up (TTFT up), preemptions up (mid-answer stalls).